



Module 6 – Administration

Superviser, analyser, dépanner, maintenir

Kubernetes – Orchestration des conteneurs

Objectifs du module

À la fin de ce module, vous saurez :

- mesurer la consommation des pods et des nœuds (**metrics-server**, `kubectl top`) et situer Prometheus et Grafana ;
- retrouver les **logs** d'un conteneur, d'un composant et d'un nœud ;
- diagnostiquer méthodiquement un pod en échec ;
- surveiller la santé des nœuds avec **Node Problem Detector** ;
- mettre un nœud en **maintenance** sans interrompre TrackR.



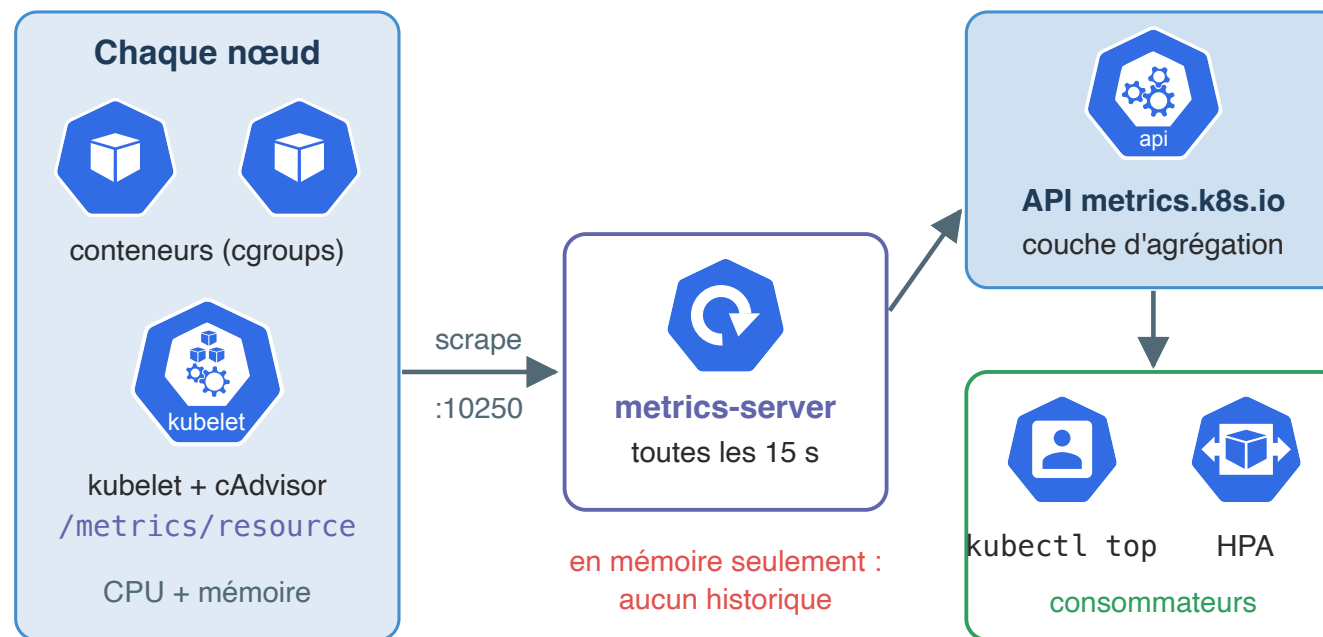
Plan du module

1. **Supervision** : pipeline de métriques, `kubectl top`, Prometheus et Grafana.
 2. **Logs** : `kubectl logs`, fichiers du nœud, composants, agrégation avec Fluent Bit.
 3. **Debugging** : méthode, états des pods, événements, `exec`, `debug`, `port-forward`, `cp`.
 4. **Les nœuds** : Node Problem Detector, StackDriver devenu Google Cloud Operations, maintenance avec `cordon`, `drain`, `uncordon`.
- Puis la démo 6 et le TP 6 : dépanner un TrackR volontairement cassé.

1. Supervision

Mesurer avant de deviner

Le pipeline de métriques



Le **kubelet** expose la consommation de chaque conteneur ; **metrics-server** la collecte et la publie via l'API `metrics.k8s.io`.

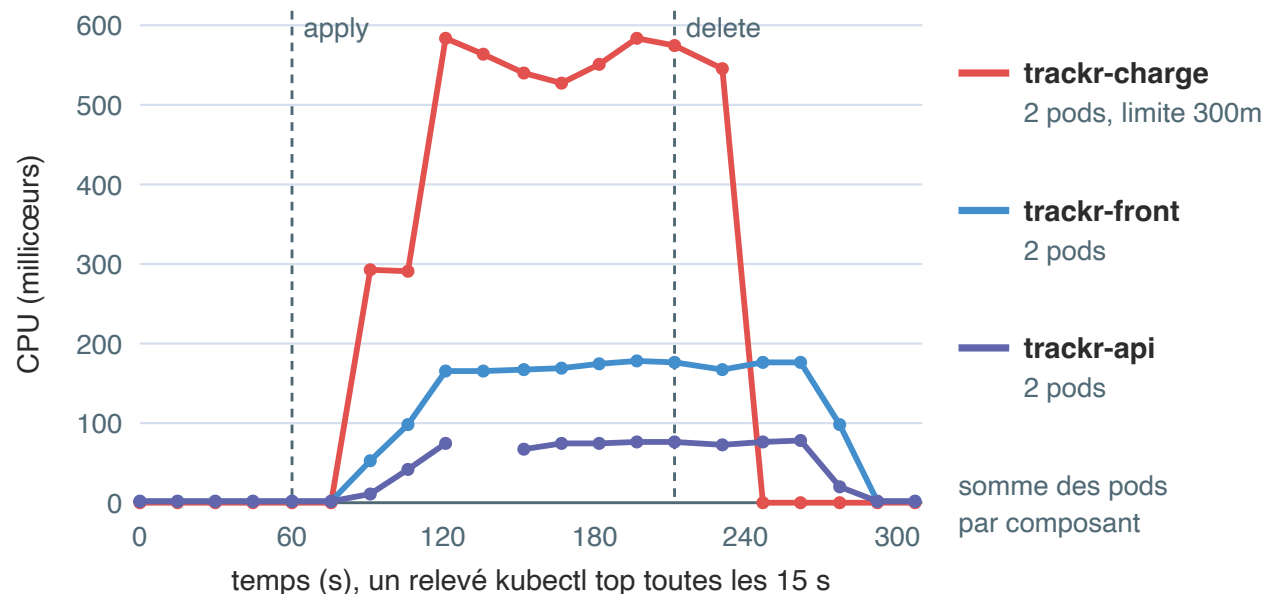
kubectl top en pratique

```
kubectl top nodes  
kubectl top pods -n trackr --sort-by=memory  
kubectl top pods -n trackr -l app=trackr-api --containers
```

NAME	CPU(cores)	MEMORY(bytes)
trackr-api-59c98686fb-mvcmk	1m	13Mi
trackr-front-855f78f74b-7cnzt	1m	13Mi
trackr-cache-6fcdcf449-s6719	3m	3Mi

- Un **instantané** sur une fenêtre de ~15 s : ni historique, ni alerte.
- Après un `delete`, un pod en arrêt disparaît de `top` alors qu'il tourne encore (délai de grâce de 30 s).

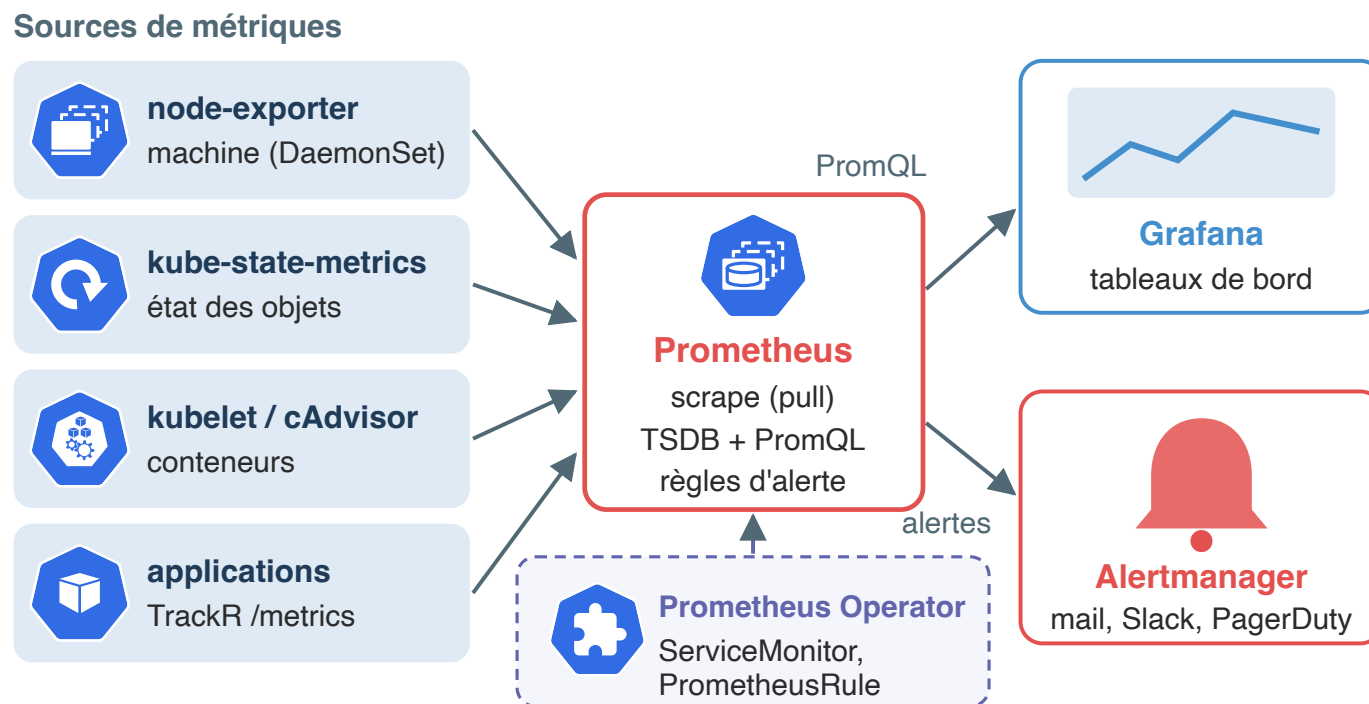
TrackR sous charge : ce que montre top



Chaque pod de charge plafonne à ~290m (**limite 300m**) ; métriques en retard de 15 à 30 s.

Mesuré sur le lab (kind v1.35, 3 nœuds) : 2 pods de charge, 4 boucles `wget` chacun. Point api absent à t = 136 s.

kube-prometheus-stack en un schéma



Installé par le chart Helm `prometheus-community/kube-prometheus-stack` (présentation, non installé sur le lab).

Quelle source pour quelle question ?

Outil	Répond à
metrics-server	« combien consomme ce pod maintenant ? »
kube-state-metrics	« combien de pods non prêts ? »
node-exporter	« le disque du nœud est-il plein ? »
Prometheus	« et depuis une semaine ? »
Grafana / Alertmanager	voir et être prévenu



2. Analyse des logs

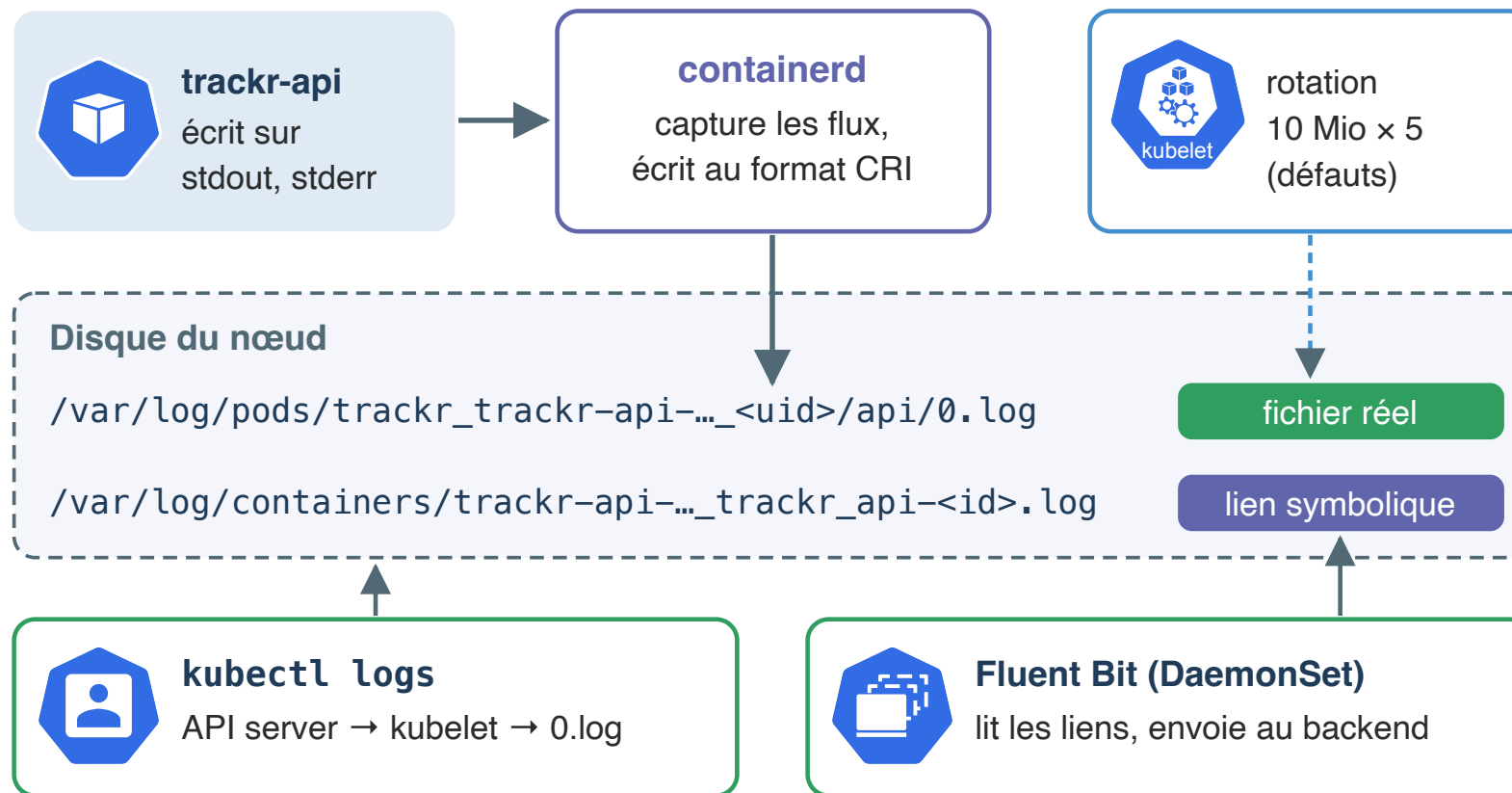
Du conteneur au backend centralisé

kubectl logs : les options utiles

```
kubectl logs -n trackr deploy/trackr-api           # un pod du Deployment
kubectl logs -n trackr -l app=trackr-api --prefix  # tous les pods, préfixés
kubectl logs -n trackr trackr-api-... -f           # suivre en continu
kubectl logs -n trackr trackr-api-... --since=10m  # fenêtre de temps
kubectl logs -n trackr trackr-api-... --tail=20    # dernières lignes
kubectl logs -n trackr trackr-api-... -c debugger  # conteneur précis
kubectl logs -n trackr trackr-worker-crash --previous # instance précédente
```

`--previous` est **le** réflexe en CrashLoopBackOff : l'instance courante n'a souvent encore rien écrit.

Où vivent les logs sur le nœud



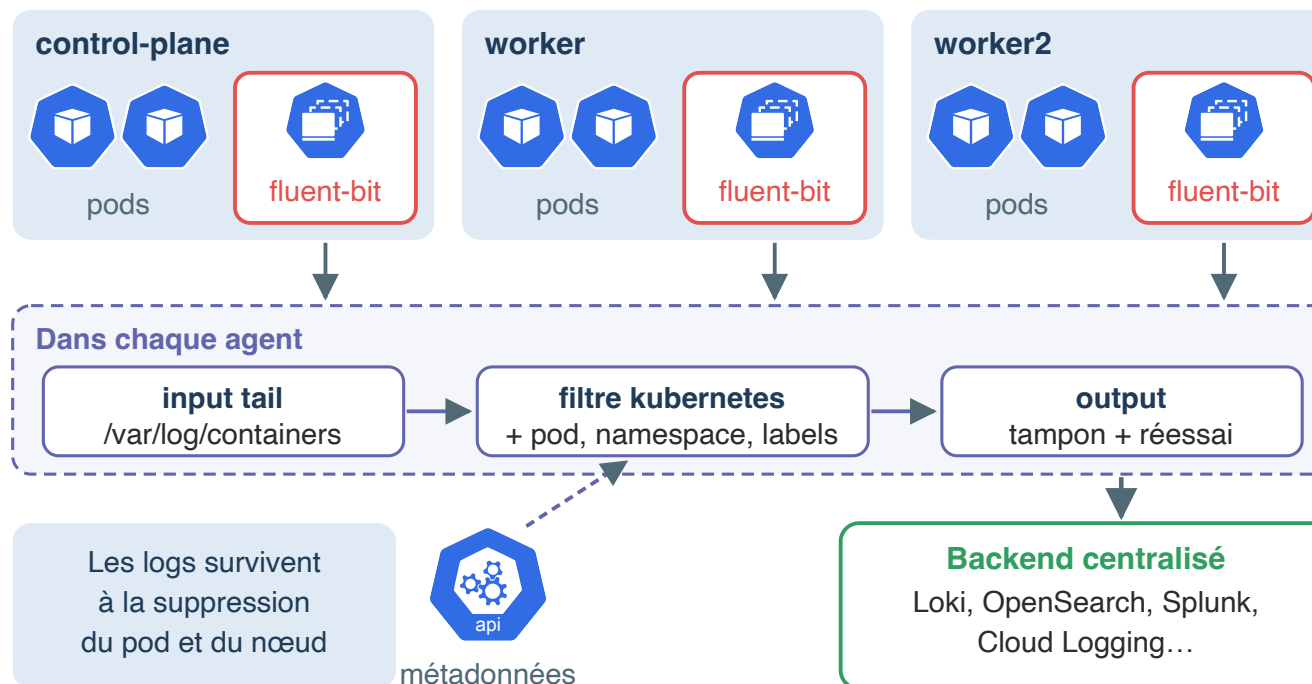
Logs du control-plane et du kubelet

```
kubect1 logs -n kube-system kube-scheduler-k8s-formation-control-plane --tail=2  
docker exec k8s-formation-worker journalctl -u kubelet --since "-5 min" \  
| grep trackr-worker-crash
```

```
I1004 20:36:55.168811 1 leaderelection.go:272] "Successfully acquired lease" ...  
kubelet[197]: ... CrashLoopBackOff: "back-off 10s restarting failed container=worker ...  
kubelet[197]: ... CrashLoopBackOff: "back-off 20s restarting failed container=worker ...
```

- API server, scheduler, controller-manager, etcd : **pods statiques**, donc `kubect1 logs`.
- kubelet et containerd : **services systemd**, donc `journalctl` sur le nœud.

Agréger : Fluent Bit en DaemonSet

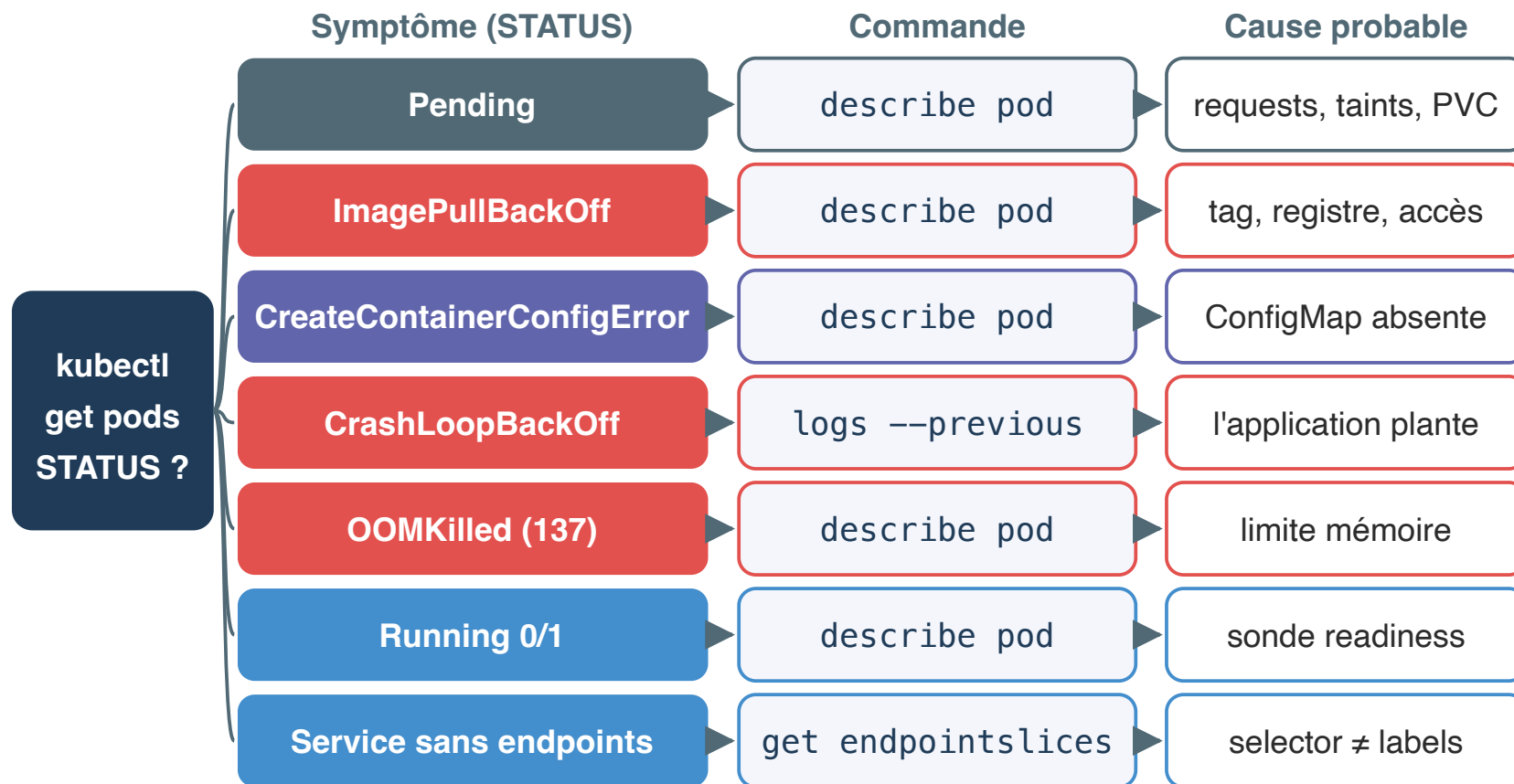


Chaque ligne reçoit pod, namespace, nœud et labels : on filtre « tous les warn de `app=trackr-api` ».

3. Debugging

Une méthode plutôt que de l'intuition

Méthode : partir du STATUS



Les états à reconnaître

STATUS	Ce qui se passe	Où regarder
Pending	aucun nœud ne convient	describe : FailedScheduling
ImagePullBackOff	image introuvable ou refusée	describe : Failed to pull image
CreateContainerConfigError	ConfigMap ou Secret référencé absent	describe : configmap "..." not found
CrashLoopBackOff	le processus se termine, relancé avec délai	logs --previous , code de sortie
OOMKilled (137)	limite mémoire dépassée	describe : Last State
Running 0/1	sonde readiness en échec	describe : Unhealthy

Sans erreur visible côté pods mais Service muet : vérifier les **EndpointSlices**.

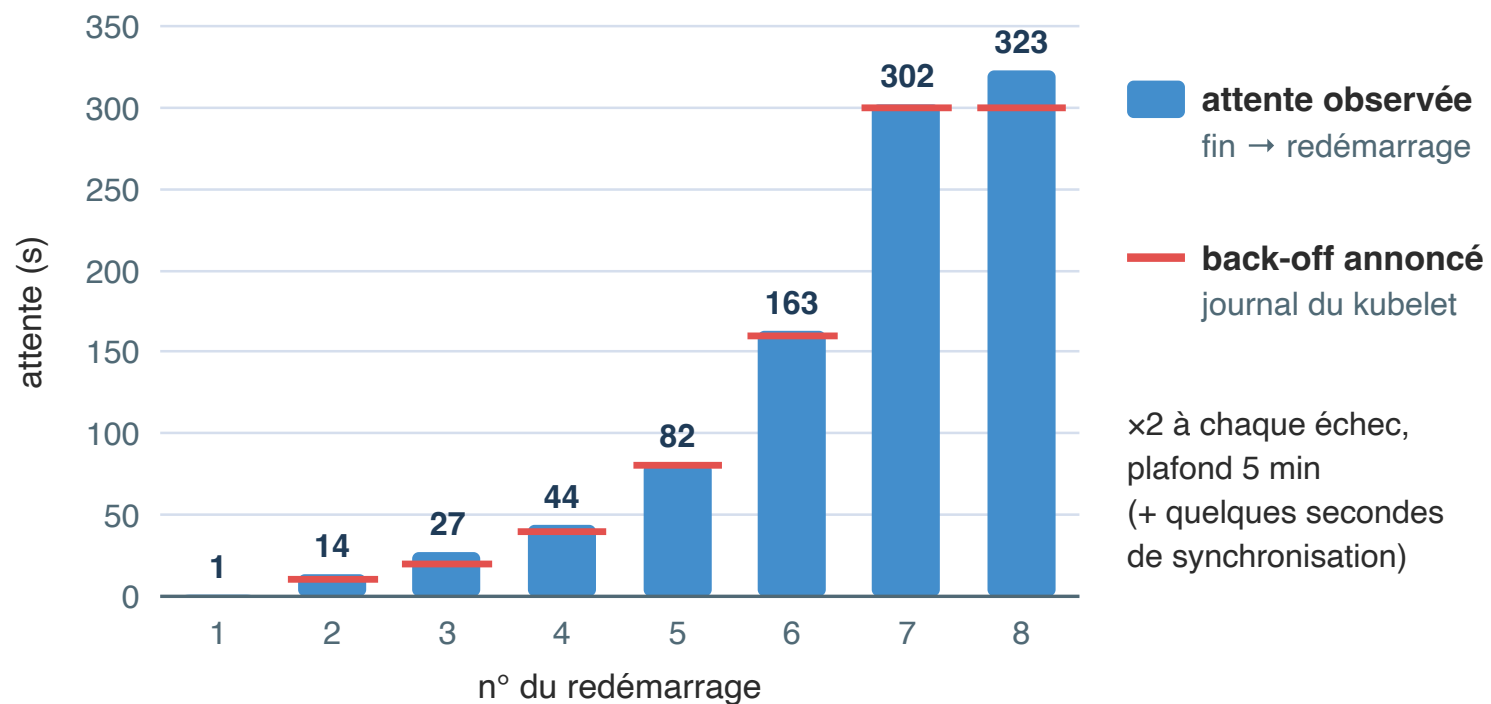
describe et événements

```
kubectl describe pod -n trackr trackr-batch-pending | sed -n '/Events:/,$p'  
kubectl get events -n trackr --sort-by=.lastTimestamp  
kubectl get events -n trackr --field-selector type=Warning
```

```
Warning FailedScheduling 12s default-scheduler 0/3 nodes are available:  
  1 node(s) had untolerated taint(s), 2 Insufficient cpu. ...  
Warning BackOff 11s kubelet Back-off restarting failed container worker ...
```

Les événements sont conservés **1 heure** par défaut : diagnostiquer vite, ou les exporter.

CrashLoopBackOff : le backoff mesuré



Mesuré sur le lab : pod busybox qui sort en code 1, 8 redémarrages en 16 min (horodatages `startedAt`).

OOMKilled : la preuve dans describe

```
kubectl describe pod -n trackr trackr-import-oom | sed -n '/State:/,/Restart/p'
```

```
State:          Terminated
Reason:         OOMKilled
Exit Code:      137
Started:        Sun, 04 Oct 2026 21:55:22 +0100
Finished:       Sun, 04 Oct 2026 21:55:22 +0100
```

- $137 = 128 + 9$: **SIGKILL** envoyé par le noyau (limite de 32 Mi).
- `kubectl logs` ne montre que « import des colis... » : l'application n'a **rien pu écrire**.
- Remède : mesurer (`kubectl top`), puis ajuster la limite ou corriger la fuite.

Service sans endpoints

Cas 1 : le sélecteur ne correspond à aucun pod



Cas 2 : le sélecteur est bon, mais les pods ne sont pas prêts



kubectl exec : un shell dans le conteneur

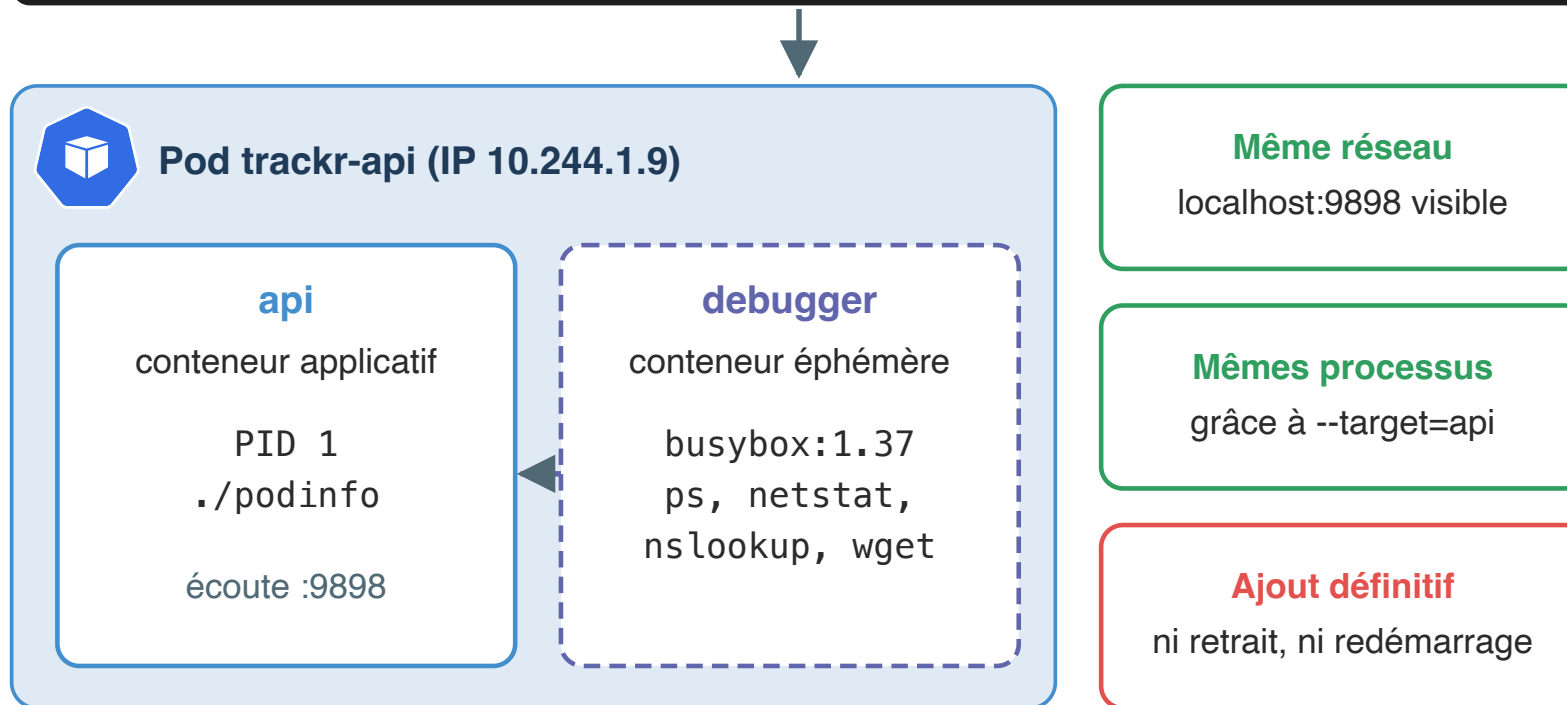
```
kubectl exec -it -n trackr deploy/trackr-api -- sh
kubectl exec -n trackr trackr-api-59c98686fb-5nww6 -- sh -c 'id; env | grep LOG_LEVEL'
kubectl exec -n trackr deploy/trackr-cache -- redis-cli ping
```

```
uid=100(app) gid=101(app) groups=101(app)
LOG_LEVEL=info
PONG
```

Limites : il faut un **shell dans l'image** (absent des images distroless) et les outils présents. Sinon, conteneur éphémère.

kubectl debug : le conteneur éphémère

```
$ kubectl debug -it trackr-api-... --image=busybox:1.37 --target=api
```



kubectl debug en pratique

```
kubectl debug -n trackr trackr-api-59c98686fb-5nww6 --image=busybox:1.37 \
  --target=api -c debugger --profile=general -- sh -c 'ps; netstat -tln'
kubectl logs -n trackr trackr-api-59c98686fb-5nww6 -c debugger
```

PID	USER	TIME	COMMAND
1	100	0:01	./podinfo --port=9898 --cache-server=tcp://trackr-cache:6379 ...
33	root	0:00	sh -c ps; ...
tcp	0	0	:::9898 :::* LISTEN

En interactif : `kubectl debug -it ... -- sh`. Variante pour un pod qui plante :

`kubectl debug <pod> --copy-to=<copie> --container=<c> -it -- sh` crée une **copie** modifiable.

Nœud, port-forward et cp

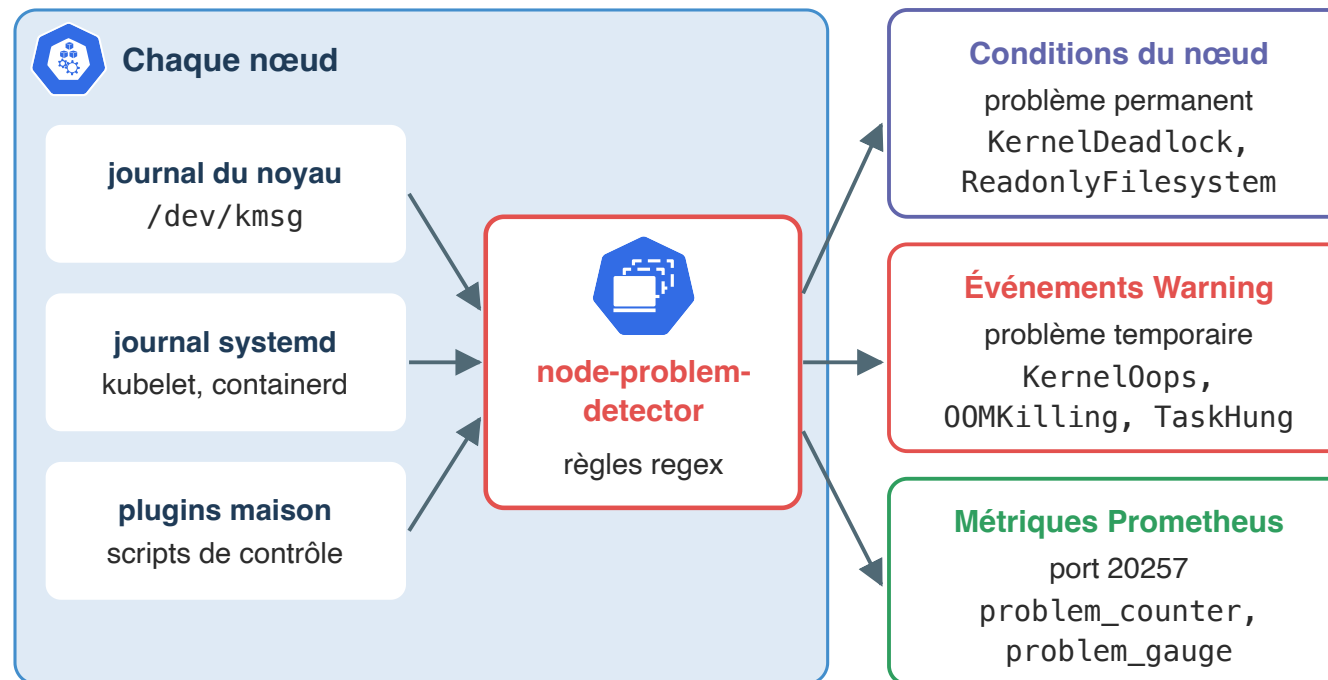
```
kubectl debug node/k8s-formation-worker -it --image=busybox:1.37 --profile=sysadmin  
# le système de fichiers du nœud est monté sous /host : chroot /host  
kubectl port-forward -n trackr svc/trackr-front 8080:80 # puis curl localhost:8080  
kubectl cp trackr/trackr-cache-...:/data/dump.rdb ./dump.rdb  
kubectl cp ./diag.txt trackr/trackr-cache-...:/tmp/diag.txt
```

- `debug node` crée un pod `node-debugger-...` (hostPID, hostNetwork) : **le supprimer** ensuite.
- `port-forward` passe par l'API server : idéal pour tester sans exposer de Service.
- `cp` s'appuie sur `tar` **dans** le conteneur.

4. Les nœuds

Santé, supervision cloud et maintenance

Node Problem Detector (NPD)



Le kubelet ignore les pannes du noyau, du disque ou du runtime : **NPD** les rend visibles dans l'API.

NPD sur le lab : injection d'un problème

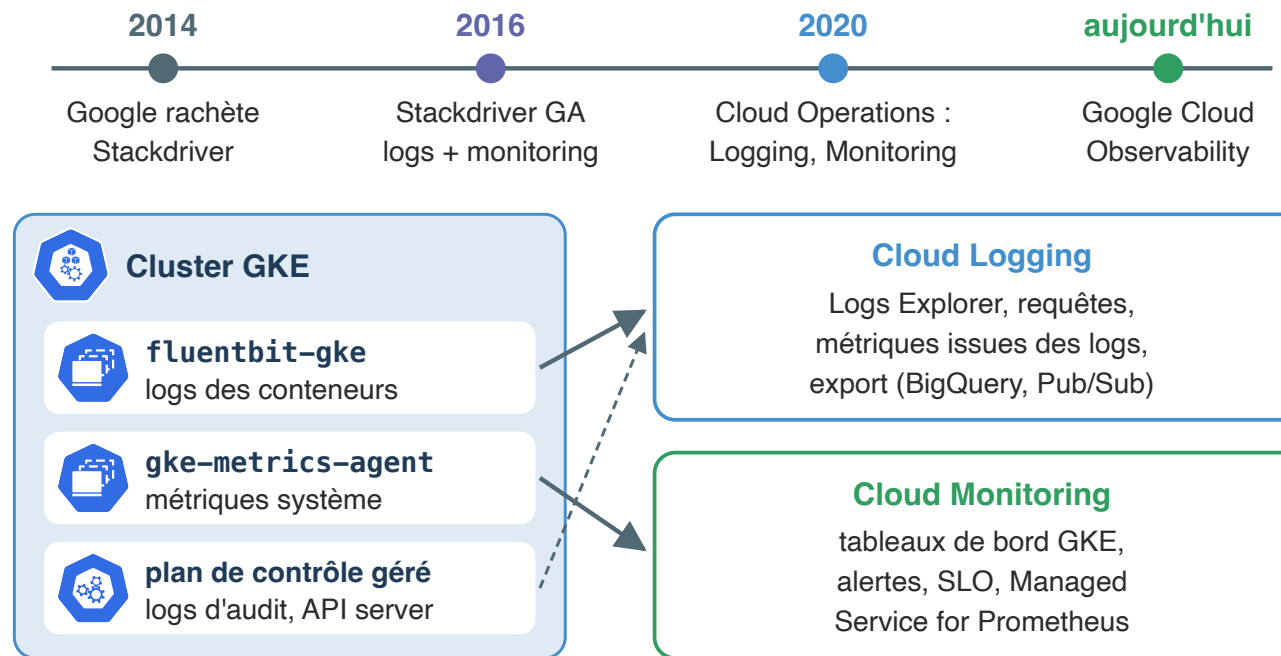
```
helm upgrade --install m06-npd deliveryhero/node-problem-detector \
  --version 2.3.14 -n m06-npd --create-namespace -f 30-npd-values.yaml
docker exec k8s-formation-worker sh -c 'echo "$(date "+%b %e %H:%M:%S") \
  k8s-formation-worker kernel: [ 1240.000001] EXT4-fs (sda1): Remounting \
  filesystem read-only" >> /var/log/m06-kern.log'
# idem avec « BUG: unable to handle kernel NULL pointer dereference at ... »
kubectl describe node k8s-formation-worker | grep -A3 Conditions
```

```
ReadonlyFilesystem True ... FilesystemIsReadOnly EXT4-fs (sda1): Remounting ...
Warning KernelOops node/k8s-formation-worker BUG: unable to handle kernel NULL ...
```

NPD : ce qu'il faut savoir

- Sur le lab (kind sous Podman rootless), `/dev/kmsg` est illisible : NPD plante avec `operation not permitted`. On lit donc un **fichier de journal simulé** (plugin `filelog`).
- Le nœud reste `Ready` : NPD **signale**, il ne **déplace** rien. La remédiation vient d'un autre outil (autoréparation GKE, Draino, scripts).
- Les conditions **persistent** après désinstallation : les retirer du statut du nœud.
- Métriques exposées : `problem_counter{reason="Kernel0ops"} 1`.

StackDriver devenu Google Cloud Operations



Présenté d'après la documentation Google Cloud : **non joué sur le lab de formation** (nécessite un projet GCP et un cluster GKE).

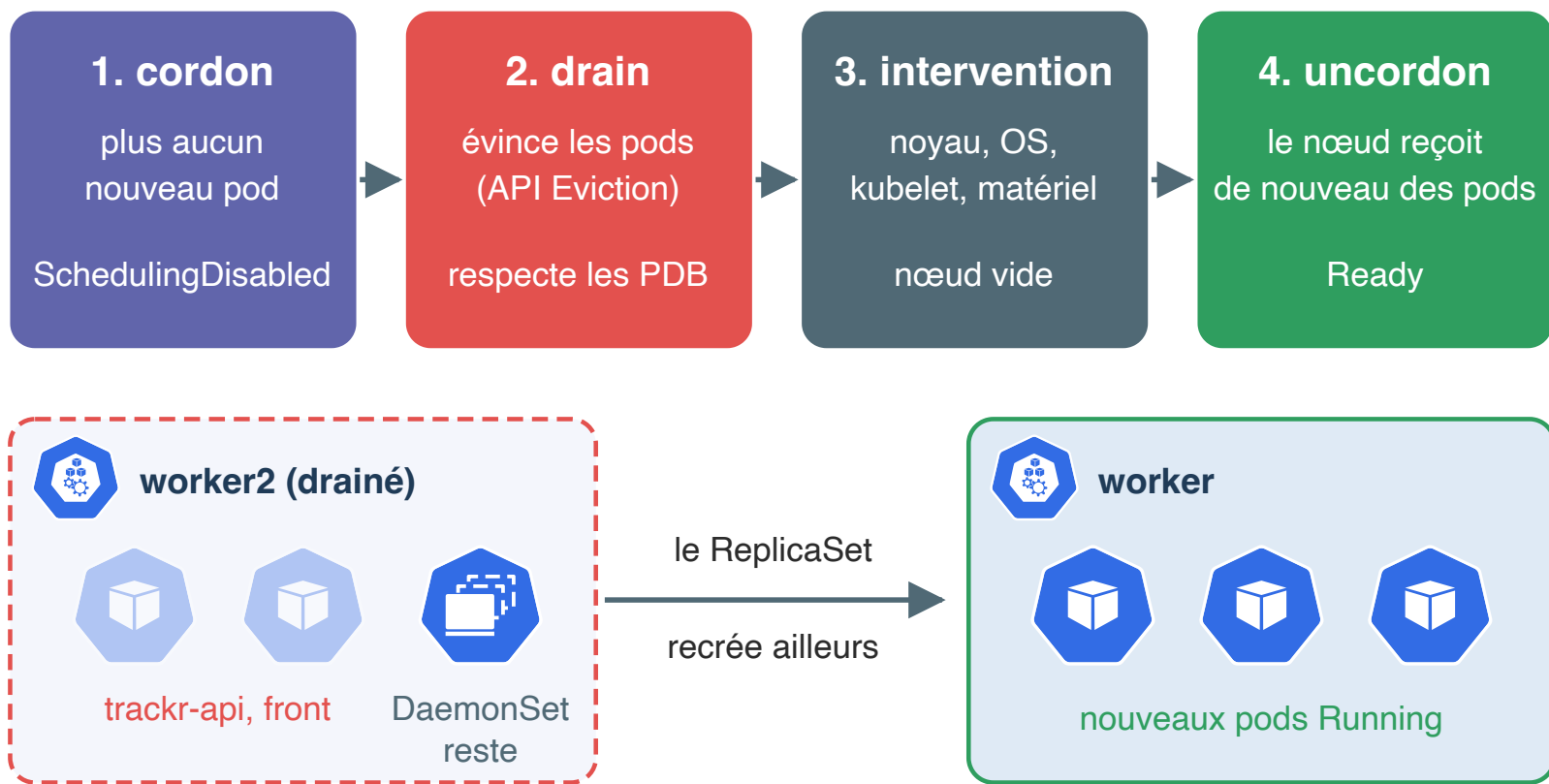
Cloud Logging et Monitoring sur GKE

```
resource.type="k8s_container"  
resource.labels.namespace_name="trackr"  
labels."k8s-pod/app"="trackr-api"  
severity>=WARNING
```

- **Logs Explorer** : requêtes comme ci-dessus, métriques dérivées des logs, export BigQuery.
- **Cloud Monitoring** : tableaux de bord GKE prêts à l'emploi (cluster, nœud, workload).
- **Alertes** : condition sur une métrique (ex. redémarrages de conteneur) et canal de notification.
- Activation par cluster : `gcloud container clusters update ... --logging=SYSTEM,WORKLOAD.`

Non rejoué sur le lab de formation.

cordon, drain, uncordon



drain en pratique (lab)

```
kubectl cordon k8s-formation-worker2  
kubectl drain k8s-formation-worker2 --ignore-daemonsets --delete-emptydir-data  
kubectl uncordon k8s-formation-worker2
```

```
node/k8s-formation-worker2 cordoned  
evicting pod trackr/trackr-front-855f78f74b-2r8nx  
evicting pod trackr/trackr-api-59c98686fb-sv5cq  
node/k8s-formation-worker2 drained  
node/k8s-formation-worker2 uncordoned
```

Sans options, `drain` refuse : cannot delete DaemonSet-managed Pods, Pods with local storage, Pods that declare no controller (use `--force`).

Lab partagé : drain limité aux pods TrackR (`--pod-selector`), nœud indisponible 5 s.

Démo 6 et TP 6

Démo — `demos/demo-06-logs-debug-npd.md` : `top` sous charge, `CrashLoopBackOff` et `backoff`, conteneur éphémère, Node Problem Detector, `drain`.

TP — `tp/tp-06-depanner-trackr.md` (45 min)

Vendredi soir, une mise en production de TrackR s'est mal passée : **rien ne répond**. Déployez le manifeste fourni dans `trackr`, diagnostiquez chaque composant avec la méthode du module, réparez, puis prouvez que `trackr-front` répond à travers son Service.

À retenir

- `kubectl top` : un instantané ; l'historique et les alertes viennent de Prometheus.
- Logs : `--previous` en crash, `journalctl` pour le kubelet, Fluent Bit pour centraliser.
- Diagnostic : **STATUS** → **describe/events** → **logs** → **exec/debug**.
- NPD rend visibles les pannes du nœud, sans les réparer.
- Maintenance : `cordon` → `drain --ignore-daemonsets` → `uncordon`.

